

Rappels :

- La fonction `malloc(n * sizeof(type))` renvoie un pointeur vers une plage mémoire pouvant stocker `n` éléments de type `type`.
Pour libérer cette plage mémoire, on appelle la fonction `free` sur un pointeur vers la plage.
- Si `p` pointe vers une plage de `n` éléments $x_0, x_1, \dots, x_{n-1}$ (de type `type`), alors `p + 1` pointe vers les `n - 1` éléments $x_1, \dots, x_{n-1}$. Plus généralement, `p + i` est égal à l'adresse du tableau de `n - i` éléments $x_i, x_{i+1}, \dots, x_{n-1}$.

Matrices

1. Ecrire un type `t_matrix`, avec `typedef struct`. Les données sont de type `float`, et la matrice sera représentée par un pointeur de type `float **`. On stocke également le nombre de lignes (`rows`) et de colonnes (`cols`) dans le struct.
2. Ecrire une fonction `create_zero_matrix`, qui renvoie une nouvelle matrice de `rows` lignes et `cols` colonnes, remplie de 0.

```
t_matrix create_zero_matrix(int rows, int cols);
```

- Cette fonction commence par allouer un tableau de taille `rows` contenant des `float *`.
 - Chaque case du tableau contient l'adresse d'une plage de taille `cols`, allouée dynamiquement.
3. Ecrire une fonction `destroy_matrix`, qui libère toute la mémoire allouée par une matrice.

```
void destroy_matrix(t_matrix *matrix);
```

4. Ecrire une fonction `get` qui renvoie un **pointeur** vers le champ `i, j` de la matrice `matrix`.

```
float *get(t_matrix *matrix, int i, int j);
```

5. Ecrire une fonction `swap_rows` qui échange les lignes `i1` et `i2` de la matrice `matrix`.

```
void swap_rows(t_matrix *matrix, int i1, int i2);
```

Quelle est la complexité de votre fonction, en fonction de `rows` et `cols` ?

Multiplication de matrices

Le produit de deux matrices **a** et **b** est défini par :

```
// a->cols == b->rows
// la matrice de sortie est de dimensions a->rows * b->cols
t_matrix matrix_mult(t_matrix *a, t_matrix *b) {
    t_matrix m = create_zero_matrix(a->rows, b->cols);

    for (int i = 0; i < a->rows; i++) {
        for (int j = 0; j < b->cols; j++) {

            float *mij = get(&m, i, j);
            for (int k = 0; k < a->cols; k++) {
                float aik = *get(a, i, k);
                float bkj = *get(b, k, j);
                (*mij) += aik * bkj;
            }

        }
    }
    return m;
}
```

6. Quelle est la complexité en temps et en espace de la fonction **matrix_mult** ?

On se propose d'implémenter *l'algorithme de Strassen*, qui est un algorithme Diviser-pour-régner qui permet d'effectuer une multiplication rapide de matrices en $O(n^{\log_2(7)}) = O(n^{2.81})$.

Le principe de l'algorithme est le suivant :

- On souhaite multiplier les matrices **A** et **B**, de taille $2^n \times 2^n$ (chacune a 2^n lignes et 2^n colonnes, pour un certain n). On écrit **C** le produit de **A** et **B**.
Si $n > 0$, on peut décomposer les matrices **A**, **B** et **C** en 4 matrices de taille $2^{n-1} \times 2^{n-1}$ chacune :

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix},$$

- On calcule alors (récursivement) les matrices M suivantes :

$$M_1 = (A_{11} + A_{22}) \times (B_{11} + B_{22});$$

$$M_2 = (A_{21} + A_{22}) \times B_{11};$$

$$M_3 = A_{11} \times (B_{12} - B_{22});$$

$$M_4 = A_{22} \times (B_{21} - B_{11});$$

$$M_5 = (A_{11} + A_{12}) \times B_{22};$$

$$M_6 = (A_{21} - A_{11}) \times (B_{11} + B_{12});$$

$$M_7 = (A_{12} - A_{22}) \times (B_{21} + B_{22}),$$

- On en déduit la matrice C :

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}.$$

7. Ecrire une fonction `get_submatrix_top_right` qui renvoie la matrice A_{12} associée à la matrice A .
On essaiera si possible de ne pas copier les valeurs de la matrice dans de nouveaux tableaux, mais simplement d'allouer un tableau de `float*` et d'y écrire les bons pointeurs. On souhaite une fonction de complexité $O(\text{rows})$.

```
t_matrix get_submatrix_top_right(t_matrix *matrix);
```

On suppose à présent les fonctions `get_submatrix_top_left`, `get_submatrix_bottom_left` et `get_submatrix_bottom_right` déjà écrites.

On suppose que l'on dispose de fonctions `add_mat` et `subtract_mat` qui calculent respectivement la somme et la différence de deux matrices de mêmes dimensions :

```
t_matrix add_mat(t_matrix *a, t_matrix *b);
t_matrix subtract_mat(t_matrix *a, t_matrix *b);
```

8. Ecrire le calcul de la matrice M_1 , telle que spécifiée dans l'algorithme de Strassen :

```
t_matrix strassen(t_matrix *a, t_matrix *b) {
    t_matrix a11 = get_submatrix_top_left(a);
    // (on suppose a12, a21, a22, b11, b12, b21 définies ici)
    t_matrix b22 = get_submatrix_bottom_right(b);

    // à compléter

    t_matrix m1 = /* à compléter */ ;

    // on suppose m2, ..., m7 calculées de la même manière
}
```

9. On dispose des matrices c_{11} , c_{12} , c_{21} et c_{22} , de taille $2^{n-1} \times 2^{n-1}$. Compléter la fonction qui calcule la matrice c de taille $2^n \times 2^n$, comme spécifié dans l'algorithme.

```
t_matrix compute_result_strassen(
    t_matrix *c11, t_matrix *c12,
    t_matrix *c21, t_matrix *c22) {

    t_matrix c =

    // à compléter

    return c;
}
```

Bonus : Complexité de l'algorithme

Nous allons calculer la complexité de l'algorithme de Strassen. On suppose que l'on multiplie des matrices de taille $N \times N$, où $N = 2^n$.

On note C_n le nombre de multiplications de flottants nécessaires pour exécuter l'algorithme sur deux matrices de taille $2^n \times 2^n$. On suppose que $C_0 = 1$.

10. Ecrire la formule récurrente qui donne C_n en fonction de C_{n-1} .

11. En déduire la valeur de C_n pour $n \geq 0$, en fonction de n .

12. En observant que $n = \log_2(N)$, montrer que la complexité de l'algorithme est :

$$C_n = N^{\log_2(7)}$$